

LAB 8 – Building an API-powered List Screen (Flutter + REST API)

1. Objective

In this lab, students will build a simple Flutter screen that retrieves data from a public REST API, parses JSON into model classes, and displays the data using ListView.

This lab reinforces the key concepts from **Module 8 – Networking & JSON**, including:

- Making network requests with the http package
- Decoding JSON responses
- Converting JSON into Dart model classes
- Displaying asynchronous data using FutureBuilder
- Showing loading and error states
- (Optional) Sending data via POST requests

At the end, students will have a fully functional API-powered list screen that runs correctly in:

- **DartPad (Flutter)**
 - **Android Studio**
 - **VS Code**
-

2. Requirements

This lab is divided into **four sub-tasks**, each focusing on one major skill:

- **Lab 8.1 – Simple GET Request**
- **Lab 8.2 – JSON → Model Conversion & ListView**
- **Lab 8.3 – Loading + Error Handling**
- **Lab 8.4 – Creating an ApiService (Service Layer Pattern)**
- (Optional) **POST request: Create a new item**

You may complete these tasks progressively in the same project.

2.1 Functional Requirements

The final API-powered screen must:

Fetch data from a public REST API

- Use a safe and stable endpoint (e.g. <https://jsonplaceholder.typicode.com/posts>)

Parse JSON properly

- Convert raw JSON list → `List<Model>`
- Use factory `Model.fromJson(Map<String, dynamic> json)`

Display API results

- Render items using `ListView` or `ListView.builder`
- Show at least 1–2 fields (e.g. title, ID)

Handle async UI states

- Loading indicator (`CircularProgressIndicator`)
- Friendly error messages (e.g. “Something went wrong”)
- Retry button (*optional*)

Optional: POST

- Simple form to send a POST request
- Display success or error

2.2 Technical Requirements

Your project must:

- Use Flutter + Dart only (no third-party network libraries)
- Use the `http` package for API calls
- Implement model parsing with `dart:convert`
- Use `FutureBuilder` to bind async data to the UI
- Use a separate **ApiService** class for networking
- Run without errors on:

- DartPad
- Android Studio
- VS Code

Required Widgets & Methods:

- FutureBuilder
 - ListView.builder
 - CircularProgressIndicator
 - json.decode()
 - factory Model.fromJson()
 - Exception handling
-

2.3 Optional Enhancements (Bonus)

Students may optionally:

- Add pull-to-refresh
- Add a detail screen when tapping an item
- Add POST request to create a new item
- Improve UI with cards, icons, or thumbnails
- Add a retry button inside the error state

These are not required but improve the final score.

3. Guided Steps

Step 1 – Project Setup

- Create a new Flutter project
(flutter create lab8_networking)
or open DartPad → New Flutter
- Replace main.dart with a minimal MaterialApp

- Verify the app runs correctly before adding logic
-

Step 2 – Call a Public API (Lab 8.1)

- Choose a sample endpoint:
`https://jsonplaceholder.typicode.com/posts`
- Create a function to fetch data using `http.get`
- Understand response format (JSON array)
- Print raw JSON for verification

Goal: retrieve API data successfully.

Step 3 – Convert JSON → Model (Lab 8.2)

- Inspect the JSON structure and extract fields
- Create a Dart model class with:
 - fields
 - constructor
 - `fromJson()` factory
- Convert JSON list → `List<Model>`

Goal: correctly parse API response into model objects.

Step 4 – Display Data Using FutureBuilder (Lab 8.2 & 8.4)

- Wrap the API call with `FutureBuilder<List<Model>>`
- Handle states:
 - waiting → loading indicator
 - `hasError` → show error
 - `hasData` → show `ListView`
- Build a clean UI with `ListTile` or custom widgets

Goal: bind async API data to UI.

Step 5 – Handle Loading + Error States (Lab 8.3)

At minimum:

- Loading spinner
- Text message for errors

(Optional):

- Retry button
- Timeout handling
- Friendly fallback UI

Goal: professional asynchronous UX.

Step 6 – Organize Networking Code into ApiService (Lab 8.4)

- Create a dedicated class ApiService
- Move all fetch logic into this class
- Inject http.Client for testing
- Keep UI clean and focused on rendering

Goal: separate concerns using the Service Layer pattern.

Step 7 – Optional POST Request

- Create a simple form for a new item (title/body)
- Send POST using http.post
- Show success or error
- Update UI or print created data

Goal: demonstrate full Read + Create workflow.

4. Expected Result

Your finished app should:

- Successfully fetch data from a REST API
- Convert JSON → Dart models
- Display the list in a scrollable widget
- Show loading state before data arrives
- Show meaningful error messages on failure
- Use a clean architecture with ApiService
- (Optional) Allow creating new items with POST

Runs in:

- DartPad
- Android Studio
- VS Code

5. Submission

Deliverables

Students must submit:

- A zipped Flutter project or
- A DartPad share link (Flutter mode)

Submission must include:

- main.dart
- ApiService class
- Model class
- Screens for list (and optional POST)

- Screenshots:
 - loading state
 - error state
 - list successfully loaded

Evaluation Criteria

Category	Weight	Description
API Integration	30%	Correct GET request, decoding, proper use of http
JSON Parsing	25%	Accurate model creation & mapping
Async UI Handling	25%	FutureBuilder, loading, error states
Architecture	10%	Clean ApiService, separation of concerns
UX Quality	10%	Readability, clean layout, friendly error messages